



Università degli Studi di Bologna
Scuola di Ingegneria e Architettura

Restituzione di valori da funzioni

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



E se il risultato non c'è...?

- In molte situazioni ci si trova di fronte a **funzioni che dovrebbero restituire un valore che però non hanno**
 - esempio: calcolo del determinante, se la matrice non è quadrata
- Come comportarsi in questi casi...?
 - per i tipi-oggetto, si può **restituire null** (con tutti i rischi del caso)
 - MA **per i tipi primitivi...?**
- Per tamponare almeno parzialmente il problema, **le due classi di libreria Float e Double** incorporano *servizi di utilità* rispettivamente per i tipi primitivi **float** o **double**
- In particolare definiscono **la costante NaN** (*Not a Number*)

Not A Number

- Le due classi **Float** e **Double** sono una sorta di «libreria matematica» per i tipi primitivi **float** o **double**, rispettivamente
- Fra le altre cose, definiscono tre *costanti pubbliche*:

MAX_VALUE	A constant holding the largest positive finite value of type float, $(2 \cdot 2^{-23}) \cdot 2^{127}$.
MIN_VALUE	A constant holding the smallest positive nonzero value of type float, 2^{-149} .
NaN	A constant holding a Not-a-Number (NaN) value of type float.
NEGATIVE_INFINITY	A constant holding the negative infinity of type float.
POSITIVE_INFINITY	A constant holding the positive infinity of type float.

- **NaN** è il valore restituito dalle forme indeterminate ($\infty - \infty$ o ∞ / ∞), o da *funzioni che non riescono a computare il risultato*



Se il risultato non c'è... NaN!

- Quindi, se il risultato non è stato calcolato, potremmo:
 - quando il tipo del risultato è un **oggetto**, restituire **null**
 - quando il tipo del risultato è **float** o **double**, restituire **NaN**
- Esempio: il determinante

```
public double det() {  
    return isSquared() ? calcDet() : risultato inesistente;  
}
```

Il risultato inesistente può ben essere rappresentato da **NaN**:

```
public double det() {  
    return isSquared() ? calcDet() : Double.NaN;  
}
```



Occhio all'operatore ternario ? :

- Si consideri la seguente funzione, che fa uso di un classico **if/else**:

```
public double det() {  
    if (isSquared()) return calcDet(); else return null;  
}
```

- Questo codice è sbagliato, in quanto **null** è assegnabile solo a riferimenti, *non certo a tipi primitivi come **double***
 - infatti, giustamente, non compila:

```
location: class Matrix  
/tmp/Veq11rV6dV/HelloWorld.java:14: error: incompatible types: <null> cannot be converted to double  
    if (isSquared()) return calcDet(); else return null;  
                                   ^
```

- MA... se invece dell' **if** usassimo l'operatore ternario ?



Occhio all'operatore ternario ? :

- Riscriviamo la funzione usando l'operatore ? :

```
public double det() {  
    return isSquared() ? calcDet() : null;  
}
```

- Questo codice è sbagliato come il precedente,
MA stavolta, incredibilmente, compila!
 - il type system viene «aggirato» perché il valore **double** restituito da **calcDet** viene *promosso automaticamente a oggetto (Double) per renderlo compatibile col null che si trova nella stessa espressione!*
 - il risultato è poi riconvertito in **double** da **return**, rendendo così «invisibile» la cosa e determinando l'apparente assurdità
- MORALE: l'operatore ternario è comodissimo e funzionale, ma ricordate che è *un'espressione* e opera come tale!



ATTENZIONE ALL'OPERATORE TERNARIO ? :

- Un semplice test

```
public static void main(String args[]) {  
    System.out.println( testTern(3<5) ); // funziona  
    System.out.println( testTern(3>5) ); // esplode a run-time  
}  
  
public static double testTern(boolean cond) { return cond ? 10 : null; }  
  
// la versione con if non compila  
// public static double testIf(boolean cond) { if(cond) return 10; else return null; }
```

**@ZioEnrico: NON
FATELO MAI !**

- E infatti..

10.0

```
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "java.lang.Integer.intValue()"
  at MyClass.testTern(MyClass.java:10)
  at MyClass.main(MyClass.java:6)
```